Şifre ve Token Hashleme

BCrypt ve SHA-256: aynı amaç, farklı yöntem. wallet projesindeki kodlarla.

> **Küçük bir düzeltme: bu "şifreleme" değil, "hashleme"**
>
> **Şifreleme (encryption)** geri açılabilir: anahtarı olan, şifreli metni tekrar düz metne çevirebilir. **Hashleme** ise **tek yönlüdür**: hash'ten asıl değere geri dönülemez. Şifreleri ve refresh token'ları **hashliyoruz**, çünkü onları hiçbir zaman geri okumamız gerekmiyor. Sadece "gelen değer doğru mu?" diye kontrol ediyoruz.

1. Ortak kural

Sırrın kendisini asla saklama, sadece hash'ini sakla. Veritabanı çalınsa bile saldırgan elindeki hash'lerle ne giriş yapabilir ne de `/refresh` isteği atabilir.

İki yöntem bu kuralı farklı şekilde uyguluyor, çünkü korudukları sırların gücü farklı:

- **Şifre:** İnsan seçer, **zayıftır** (`123456` , `murat1990`). Ağır koruma gerekir.
- **Refresh token:** Bilgisayar üretir, **güçlüdür** (256 bit rastgele). Hafif koruma yeter, ama DB'de **aranabilmesi** gerekir.

2. Özellikler yan yana

	Şifre — BCrypt	Refresh token — SHA-256
Salt (rastgele ek değer)	Var, her seferinde yeni	Yok
Aynı girdi → aynı hash?	❌ Hayır	✅ Evet
Hız	Bilerek yavaş (~100 ms, 2^{10} tur)	Çok hızlı (tek tur, mikrosaniyeler)
Çıktı	60 karakter: `$2a$10$...`	64 karakter, hex: `3e8f45...4111`
Salt nerede saklanıyor?	Hash'in kendi içinde	Salt yok
Nasıl karşılaştırılır?	`passwordEncoder.matches(düz, hash)`	`findByTokenHash(hash(token))`
Önce kayıt bulunmalı mı?	Evet: önce kullanıcı adıyla kişi bulunur	Hayır: hash ile doğrudan aranır
DB kolonu	`users.password`	`refresh_tokens.token_hash`
Projede nerede?	`AuthService` : register, login	`RefreshTokenService` : create, rotate, revoke

3. Nasıl yapılıyor?

🔐 Şifre (BCrypt)

Araç (`SecurityConfig`)

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Kayıtta: hashle (`AuthService.register`)

```
user.setPassword(

passwordEncoder.encode(request.password()));
// "123" → "$2a$10$N9qo...lhWy"
```

Girişte: karşılaştır (`AuthService.login`)

```
// ① önce kişiyi bul
User user = userRepository
    .findByUserName(request.userName())
    .orElseThrow(...);

// ② sadece onun hash'iyle karşılaştır
passwordEncoder.matches(
    request.password(), user.getPassword());
```

🎫 Refresh token (SHA-256)

Hash metodu (`RefreshTokenService`)

```
private String hash(String rawToken) {
  MessageDigest d =
      MessageDigest.getInstance("SHA-256");
  return HexFormat.of().formatHex(
      d.digest(rawToken.getBytes(UTF_8)));
}
```

Oluştururken: hashle (`create`)

```
String raw = generateRawToken(); //
SecureRandom
rt.setTokenHash(hash(raw));      // DB'ye hash
return raw;                      // telefona
ham
```

Yenilemede: hashle ve ara (`rotate`)

```
// tek adımda: hash'le ve doğrudan bul
RefreshToken current = refreshTokenRepository
    .findByTokenHash(hash(rawToken))
    .orElseThrow(
        InvalidRefreshTokenException::new);
// SQL: WHERE token_hash = '3e8f45...'
```

BCrypt hash'inin içinde ne var?

```
$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
└┘└┘└┘└────────────────────┘└──────────────────────────────┘
 │  │  │   salt (22 karakter)          asıl hash (31 karakter)
 │  │  └─ maliyet: 10 → 2¹⁰ = 1024 tur
 │  └──── algoritma sürümü

matches("123", yukarıdaki hash):
  1. hash'ten salt'ı ve maliyeti OKU
  2. "123"'ü aynı salt + aynı maliyetle hashle
  3. sonuç kayıtlı hash'le aynı mı? → evet = şifre doğru
```

Aynı girdiye ne oluyor?

```
BCrypt   "123" (kullanıcı A)  →  $2a$10$X1k...aaa     farklı salt → FARKLI hash
         "123" (kullanıcı B)  →  $2a$10$p9Q...bbb

SHA-256  "qQxz_UIt..." (login'de)   →  3e8f453e...994111   salt yok → HEP AYNI hash
         "qQxz_UIt..." (refresh'te) →  3e8f453e...994111   → WHERE token_hash = ? ile bulunur
```

4. Neden böyle? (Salt ve yavaşlık neyi önler?)

Koruma	Şifrede gerekli mi?	Refresh token'da gerekli mi?
Yavaşlık Deneme-yanılmayı pahalı yapar	✅ **Evet.** Saldırgan yaygın şifreleri dener. Yavaşlık saniyede milyarlarca denemeyi birkaç bine düşürür.	❌ **Hayır.** $2^{256} \approx 10^{77}$ ihtimal var. En hızlı hash ile bile taranamaz.
Salt: aynı girdi gizleme Aynı şifreli kullanıcılar belli olmasın	✅ **Evet.** Birçok kullanıcı aynı şifreyi seçer.	❌ **Hayır.** İki rastgele token'ın aynı olma ihtimali yok.
Salt: hazır tablolar Önceden hesaplanmış hash listeleri	✅ **Evet.** `123456`, `qwerty` gibi şifrelerin hash tabloları hazır bulunuyor.	❌ **Hayır.** Rastgele 256 bitlik değerlerin tablosu yapılamaz.
Aranabilirlik `WHERE hash = ?`	Gerekmez: Kişi önce kullanıcı adıyla bulunur.	✅ **Şart.** Refresh isteğinde sadece token geliyor, başka arama bilgisi yok.

5. Yanlış olanı seçseydik?

Şifreyi SHA-256 ile hashlemek

Güvensiz. DB çalınırsa `123456` gibi şifreler saniyeler içinde kırılır. Aynı şifreyi kullananlar da aynı hash'e sahip olduğu için hemen fark edilir.

Refresh token'ı BCrypt ile hashlemek

Çalışmaz. Her seferinde farklı hash çıktığı için `WHERE token_hash = ?` araması yapılamaz. Her refresh'te bütün tabloyu tek tek denemek gerekir: 10.000 satır × 100 ms ≈ 16 dakika.

6. Tek bakışta

```
ŞİFRE (BCrypt)                          REFRESH TOKEN (SHA-256)
──────────────────────────────          ──────────────────────────────
Zayıf sır (insan seçer)                 Güçlü sır (SecureRandom, 256 bit)
Rastgele salt + 1024 tur (yavaş)        Tek tur, salt yok (hızlı)
Aynı şifre → farklı hash                Aynı token → aynı hash
① findByUserName  ② matches()           ① hash(token)  ② findByTokenHash()
users.password                          refresh_tokens.token_hash
```

Tek cümleyle

Zayıf bir sırrı (şifre) korumak için **yavaş ve tuzlu** BCrypt kullanılır. Güçlü bir sırrı (rastgele token) korumak için **hızlı ve tuzsuz** SHA-256 yeterlidir, üstelik veritabanında aranabilmek için zorunludur. İkisinde de sırrın kendisi değil, sadece hash'i saklanır.